1. AFTER INSERT — Audit

Purpose: Records when a new student is inserted.

```sql
CREATE DATABASE College;
USE College;

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

CREATE TABLE Student_Audit (
    Audit_ID INT AUTO_INCREMENT PRIMARY
KEY,
    Student_ID INT,
    Action VARCHAR(20)
);

DELIMITER //

CREATE TRIGGER student_insert_audit
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_Audit
    (Student_ID, Action)
    VALUES (NEW.Student_ID, 'INSERT');
END //

DELIMITER ;

INSERT INTO Student
VALUES (101, 'Rahul', 85);

SELECT * FROM Student;
SELECT * FROM Student_Audit;
```

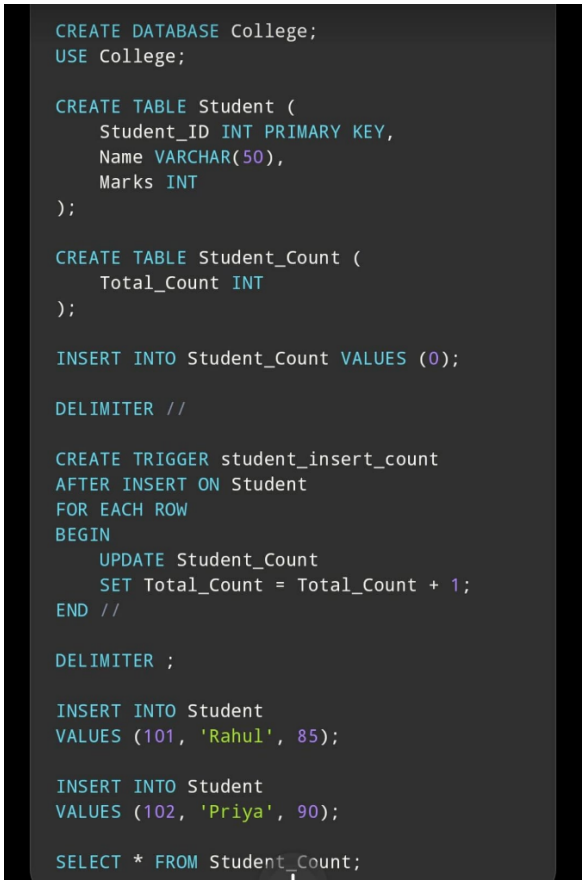
Output — Student_Audit

Audit_ID	Student_ID	Action
1	101	INSERT

---

2. AFTER INSERT — Update Count

Purpose: Automatically increases the student count after each insertion.



```sql
CREATE DATABASE College;
USE College;

CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

CREATE TABLE Student_Count (
    Total_Count INT
);

INSERT INTO Student_Count VALUES (0);

DELIMITER //

CREATE TRIGGER student_insert_count
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    UPDATE Student_Count
    SET Total_Count = Total_Count + 1;
END //

DELIMITER ;

INSERT INTO Student
VALUES (101, 'Rahul', 85);

INSERT INTO Student
VALUES (102, 'Priya', 90);

SELECT * FROM Student_Count;
```

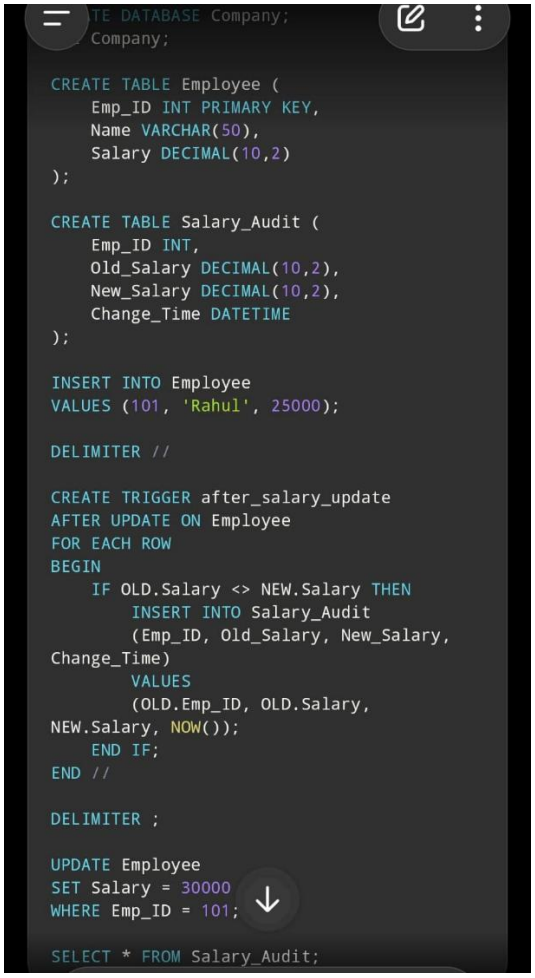
Output — Student_Count

Total_Count
2

---

3. AFTER UPDATE — Salary Audit

Purpose: Records the old salary and new salary whenever an employee's salary changes.



```
TE DATABASE Company;
 Company;

CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

CREATE TABLE Salary_Audit (
    Emp_ID INT,
    Old_Salary DECIMAL(10,2),
    New_Salary DECIMAL(10,2),
    Change_Time DATETIME
);

INSERT INTO Employee
VALUES (101, 'Rahul', 25000);

DELIMITER //

CREATE TRIGGER after_salary_update
AFTER UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary <> NEW.Salary THEN
        INSERT INTO Salary_Audit
        (Emp_ID, Old_Salary, New_Salary,
Change_Time)
        VALUES
        (OLD.Emp_ID, OLD.Salary,
NEW.Salary, NOW());
    END IF;
END //

DELIMITER ;

UPDATE Employee
SET Salary = 30000
WHERE Emp_ID = 101;

SELECT * FROM Salary_Audit;
```

Output — Salary_Audit

Emp_ID	Old_Salary	New_Salary	Change_Time
101	25000.00	30000.00	Current date and time

---

4. BEFORE UPDATE — Error Message

Purpose: Prevents an employee's salary from being updated to a negative value.

```sql
CREATE DATABASE Company;
USE Company;

CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

INSERT INTO Employee
VALUES (101, 'Rahul', 25000);

DELIMITER //

CREATE TRIGGER before_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.Salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Error: Salary cannot be negative';
    END IF;
END //

DELIMITER ;

UPDATE Employee
SET Salary = -5000
WHERE Emp_ID = 101;

SELECT * FROM Employee;
```

Output

Error: Salary cannot be negative

The salary remains 25000 because the update is rejected.

---

5. BEFORE DELETE — Error Message

Purpose: Prevents deletion of an employee whose salary is ₹50,000 or more.

```sql
CREATE DATABASE Company;
USE Company;

CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

INSERT INTO Employee
VALUES (101, 'Rahul', 60000);

DELIMITER //

CREATE TRIGGER before_employee_delete
BEFORE DELETE ON Employee
FOR EACH ROW
BEGIN
    IF OLD.Salary >= 50000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Error: Cannot delete employee';
    END IF;
END //

DELIMITER ;

DELETE FROM Employee
WHERE Emp_ID = 101;

SELECT * FROM Employee;
```

Output

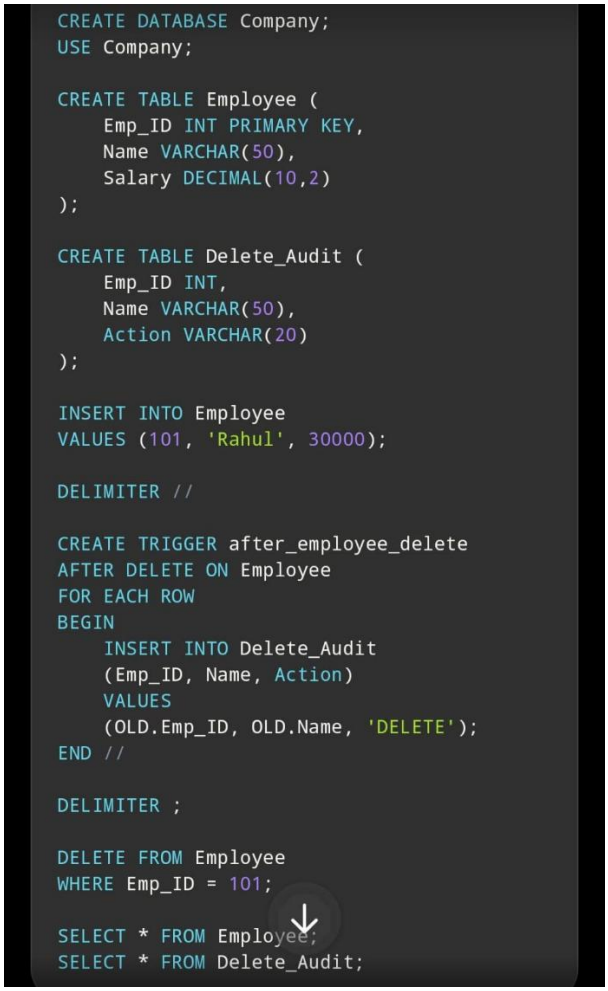
Error: Cannot delete employee

The employee is not deleted because the salary is ₹60,000.

---

6. AFTER DELETE — Audit

Purpose: Records the details of an employee after deletion.



```sql
CREATE DATABASE Company;
USE Company;

CREATE TABLE Employee (
    Emp_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);

CREATE TABLE Delete_Audit (
    Emp_ID INT,
    Name VARCHAR(50),
    Action VARCHAR(20)
);

INSERT INTO Employee
VALUES (101, 'Rahul', 30000);

DELIMITER //

CREATE TRIGGER after_employee_delete
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Delete_Audit
    (Emp_ID, Name, Action)
    VALUES
    (OLD.Emp_ID, OLD.Name, 'DELETE');
END //

DELIMITER ;

DELETE FROM Employee
WHERE Emp_ID = 101;

SELECT * FROM Employee;
SELECT * FROM Delete_Audit;
```

Output — Delete_Audit

Emp_ID	Name	Action
101	Rahul	DELETE